

torch.multinomial#

<https://pytorch.org/docs/stable/generated/torch.multinomial.html>

Description:

Returns a tensor where each row contains `num_samples` indices sampled from the multinomial (a stricter definition would be multivariate, refer to `torch.distributions.multinomial.Multinomial` for more details) probability distribution located in the corresponding row of tensor input. The rows of input do not need to sum to one (in which case we use the values as weights), but must be non-negative, finite and have a non-zero sum. Indices are ordered from left to right according to when each was sampled (first samples are placed in first column). If input is a vector, out is a vector of size `num_samples`.

Function Signature

```
torch.multinomial(input, num_samples, replacement=False, *,  
generator=None, out=None) → LongTensor#
```

Parameters

Keyword Arguments

Parameters

Keyword

generator (torch.Generator, optional) – a pseudorandom number generator for sampling out (Tensor, optional) – the output tensor.

Code Examples

```
>>> weights = torch.tensor([0, 10, 3, 0], dtype=torch.float) #  
create a tensor of weights  
>>> torch.multinomial(weights, 2)  
tensor([1, 2])  
>>> torch.multinomial(weights, 5) # ERROR!  
RuntimeError: cannot sample n_sample > prob_dist.size(-1)  
samples without replacement  
>>> torch.multinomial(weights, 4, replacement=True)  
tensor([ 2,  1,  1,  1])
```

Notes & Warnings

NOTE

Note The rows of input do not need to sum to one (in which case we use the values as weights), but must be non-negative, finite and have a non-zero sum.

NOTE

Note When drawn without replacement, `num_samples` must be lower than number of non-zero elements in input (or the min number of non-zero elements in each row of input if it is a matrix).

Detailed Documentation

Documentation

Returns a tensor where each row contains `num_samples` indices sampled from the multinomial (a stricter definition would be multivariate, refer to `torch.distributions.multinomial.Multinomial` for more details) probability distribution located in the corresponding row of tensor input.

The rows of input do not need to sum to one (in which case we use the values as weights), but must be non-negative, finite and have a non-zero sum.

Indices are ordered from left to right according to when each was sampled (first samples are placed in first column).

If input is a vector, out is a vector of size num_samples.

If input is a matrix with m rows, out is an matrix of shape $(m \times \text{num_samples}) \times m$.

If replacement is True, samples are drawn with replacement.

If not, they are drawn without replacement, which means that when a sample index is drawn for a row, it cannot be drawn again for that row.

When drawn without replacement, num_samples must be lower than number of non-zero elements in input (or the min number of non-zero elements in each row of input if it is a matrix).

input (Tensor) – the input tensor containing probabilities

num_samples (int) – number of samples to draw

replacement (bool, optional) – whether to draw with replacement or not

generator (torch.Generator, optional) – a pseudorandom number generator for sampling

out (Tensor, optional) – the output tensor.